# Fixit Linter + AI Coding

Python Code Quality Strategy for the AI Era

Python Asia 2026

# About Me

- **Name**: Naohide Anahara
- **GitHub**: github.com/naohide
- Python Developer
- Working at Tokyo Gas Co., Ltd.

# Agenda

**Part 1** Fixit Linter due to AI Coding

**01** Background & Challenges - Why Custom Linters Are Needed

**02** What Is Fixit - A libcst-Based Lint Framework

**03** AST vs CST - Why CST Matters

**04** AI-Powered Rule Generation

**05** Live Demo - Automating logging to structlog Migration

# Agenda (cont.)

**Part 2** Fixit Linter for AI Coding

**06** Quality Challenges of AI-Generated Code

**07** Guarding AI Code Quality with Fixit

**08** mutmut - Verifying Test Quality with Mutation Testing

**09** CI/CD Integration & Operations

# Part 1: Fixit Linter due to AI Coding

Custom Linters Matter More Than Ever in the AI Era

# 01. Background & Challenges

Why Do We Need Custom Linters?

How does your team enforce coding standards?

# The Reality of Code Reviews

## Common Problems

- Team-specific conventions become tacit knowledge
- Review quality depends on individual reviewers
- Review feedback tends to be subjective
- Slow onboarding of new members to conventions

## The Ideal State

- Coding standards are explicitly documented
- Conventions are verified mechanically
- Objective and consistent feedback
- Automatically enforced via CI/CD

# A Common Scenario in Practice

```
# During a PR review...

# Reviewer A: "Please use structlog instead of logging"
import logging
logger = logging.getLogger(__name__)

# Reviewer B: "Oh, I didn't know about that..."
# Reviewer C: "This wasn't flagged in the last PR..."
```

The same feedback repeated over and over = an opportunity for automation

# 3 Problems Custom Linters Solve

1

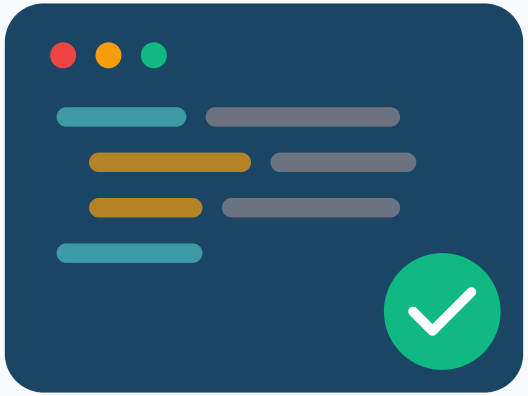
## Eliminating Subjectivity

Define coding conventions as machine-enforceable rules to prevent reviewer-dependent reviews

2

## Enforcing Best Practices

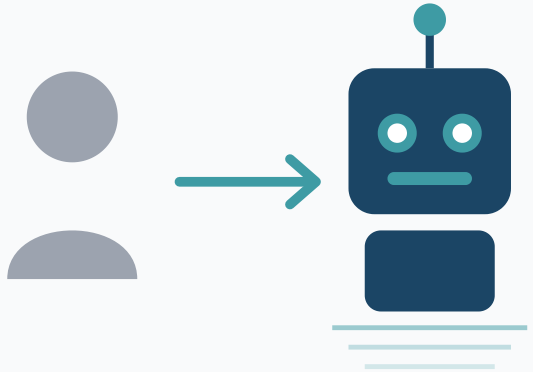
Automatically apply the organization's "ideal patterns" across the entire codebase



3

## From Human to Machine

Shift from human-dependent review processes to automated machine verification



# Limitations of General-Purpose Linters

**ruff, flake8, pylint** and other general-purpose linters are excellent, but...

- They only provide generic rules
- Cannot address organization-specific requirements
- e.g., "Use structlog instead of the logging module"
- e.g., "Always apply a specific decorator"
- e.g., "Prohibit usage of deprecated APIs"

**Custom rules are essential to enforce organization-specific coding standards**

# Examples of Organization-Specific Rules

## Rules that general-purpose linters cannot detect

Enforcing **logging** → **structlog** migration
Prohibiting direct use of **datetime.now()** (for testability)
Requiring internal HTTP client instead of **requests**
Prohibiting custom exceptions that don't inherit from a specific base class
Detecting code that calls deprecated internal APIs

# 02. What Is Fixit

A Python Lint Framework Born at Instagram

# Fixit Overview

## 🔧 Instagram/Fixit

A powerful Python lint framework based on **libcst**. Easy to create custom rules with built-in autofix capabilities.

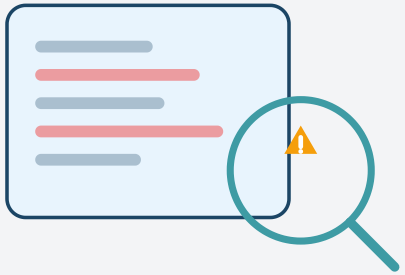
- Precise syntax analysis using **libcst** (CST = Concrete Syntax Tree)
- Place custom rules directly in your repository
- Interactive review or bulk autofix of suggested fixes
- Hierarchical configuration management (project/directory/file level)

# Key Features of Fixit

## Lint

### Detection

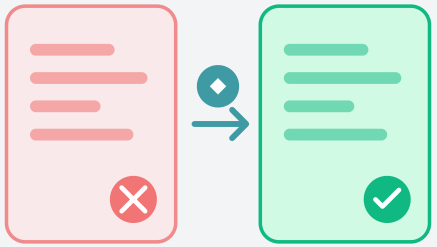
Detect and report rule violations



## Fix

### Autofix

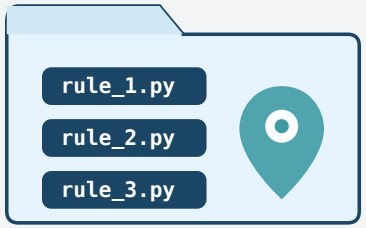
Automatically fix detected issues



## Local

### Local Rules

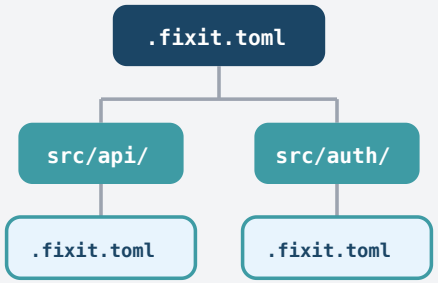
Place rules in your repository




## Config

### Hierarchical Config

Per-directory configuration



# Basic Usage of Fixit

```
# Configure in .fixit.toml
[fixit]
local = ["lint_rules/"]    # Custom rules directory

# Run commands
$ pip install fixit                    # Install
$ fixit lint path/to/code.py           # Run linting
$ fixit fix path/to/code.py            # Autofix
$ fixit fix --interactive path/        # Interactive fix
```

# Basic Structure of a Fixit Rule

```python
import libcst as cst
import fixit

class MyRule(fixit.LintRule):
    """Description of the rule"""
    MESSAGE = "Description of the issue"

    def visit_Call(self, node: cst.Call) -> None:
        """Visit specific nodes for inspection"""
        # Inspect node and report violation
        self.report(node, message="Fix required")

    def visit_Import(self, node: cst.Import) -> None:
        """Inspect import statements"""
        # Specifying replacement enables autofix
        self.report(node, replacement=new_node)
```

**Visitor Pattern**: Traverses each node in the tree for inspection

# Fixit Configuration File

```toml
# .fixit.toml - Place in the project root

[fixit]
# Custom rules directory
local = ["lint_rules/"]

# Rules to enable
enable = ["fixit.rules", "lint_rules"]

# Rules to disable
disable = ["fixit.rules:CompareSingletonPrimitivesByIs"]

# Python version
python-version = "3.14"
```

Place a separate `.fixit.toml` in subdirectories for **hierarchical configuration**

# 03. AST vs CST

Why the Concrete Syntax Tree Matters

# Two Approaches to "Understanding" Source Code

AST (Abstract Syntax Tree) vs CST (Concrete Syntax Tree)

# AST vs CST: The Difference

## AST (Abstract Syntax Tree)

- Retains only structural information
- Comments are lost
- Whitespace and newlines are lost
- Formatting is lost

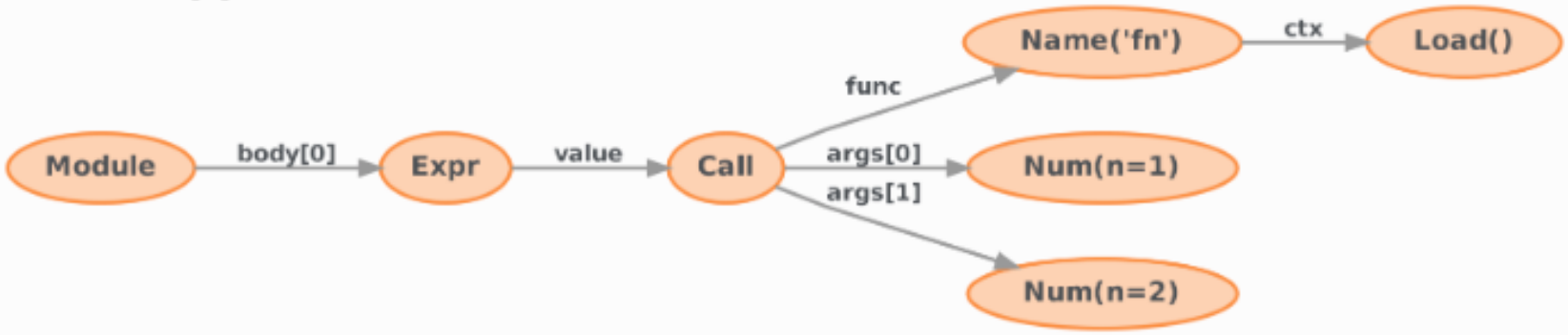
## CST (Concrete Syntax Tree)

- Retains structure + formatting info
- Comments are preserved
- Whitespace and newlines are preserved
- Original formatting is maintained

# AST Basics

```
fn(1, 2)  # calls fn
```

Show Code [+]




AST extracts only the "meaning" of code and discards the "appearance"

# The AST Problem: Information Lost During Transformation

```
# Original code
import logging  # Legacy logging

# Application's main logger
logger = logging.getLogger(__name__)
```

### When transformed with AST...

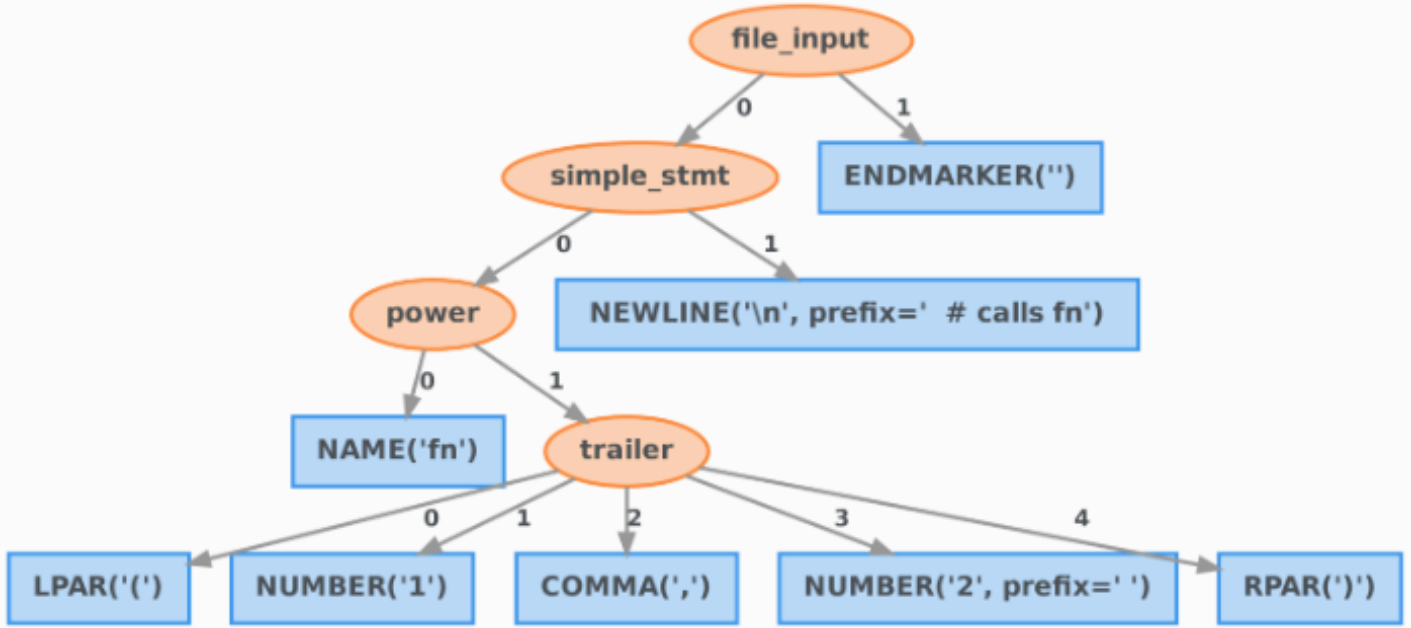
```
import structlog
logger = structlog.get_logger()
```

Comments are lost, along with the team's notes and intentions

# CST Basics

```
fn(1, 2)  # calls fn
```

Show Code [+]




CST represents code in a "fully recoverable form"

However, lib2to3 was removed in Python 3.13 → a successor is needed

# The CST Advantage: Fully Preserving Original Style

```
# Original code
import logging  # Legacy logging

# Application's main logger
logger = logging.getLogger(__name__)
```

### When transformed with CST...

```
import structlog  # Legacy logging

# Application's main logger
logger = structlog.get_logger()
```

Safe transformation while preserving comments, whitespace, and formatting

## However, lib2to3-Based CST Has Its Own Issues

- Ambiguous ownership of whitespace and comments, leading to invalid code when deleting nodes
- Manual management of nodes is required, making complex transformations cumbersome

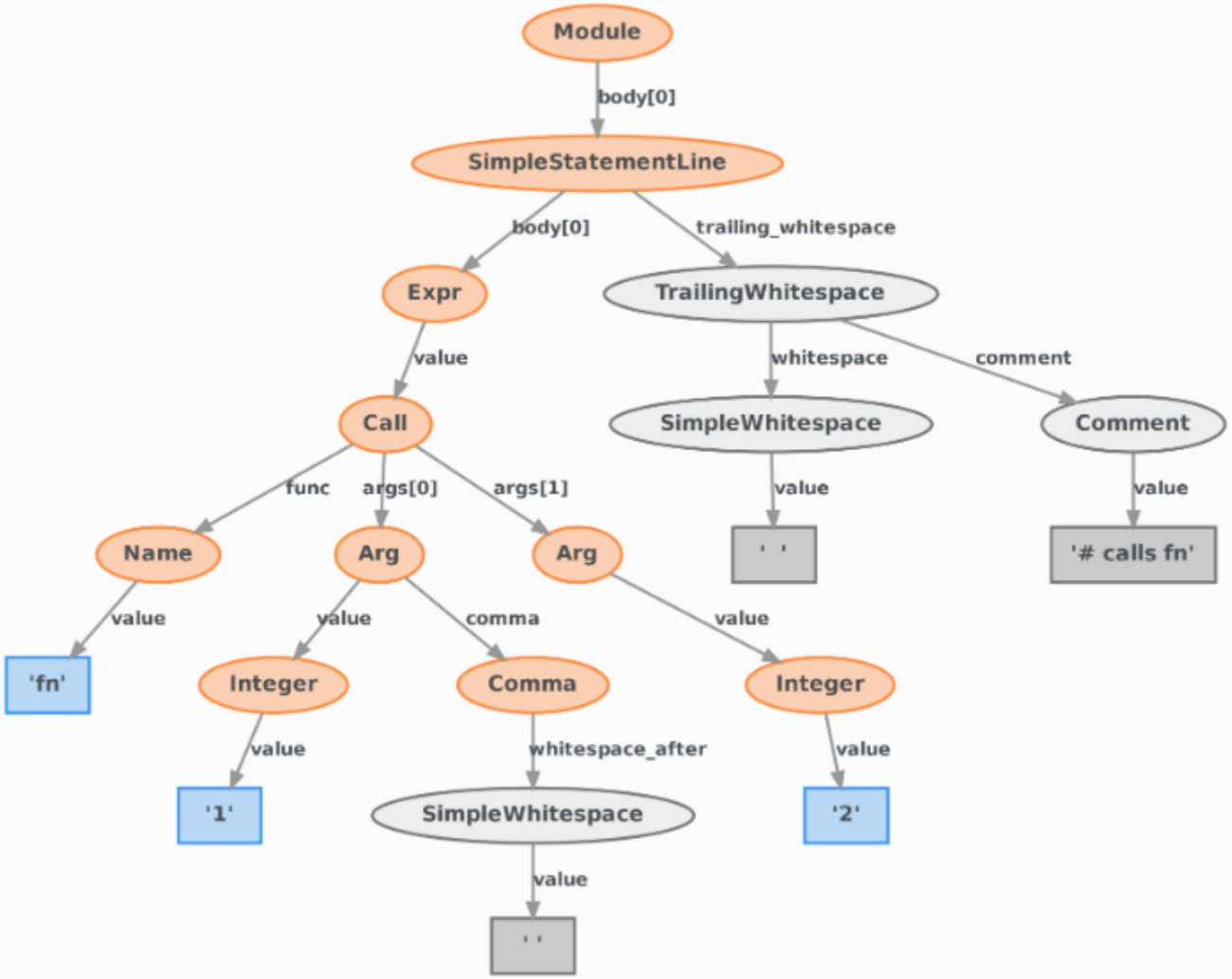
# Enter LibCST

A CST library from Meta (Instagram) that solves the issues of lib2to3

- **LibCST's Improvement**: Combines CST completeness with AST-like semantic usability
- Code transformation framework based on the Visitor pattern
- Safe immutable node operations via `with_changes()`
- Used as the foundational library for Fixit

```
fn(1, 2)  # calls fn
```

Show Code [+]



# Why LibCST Matters in Production

### "Safe automatic transformation" is essential for production code

When performing automated refactoring on large codebases, **losing comments or formatting is unacceptable**.

With LibCST, you can **transform code reliably while preserving the existing code style**, resulting in clean diffs for review.

# Clean git diffs

```
# CST-based transformation keeps diffs minimal
- import logging
+ import structlog

# Application's main logger   ← Comment preserved
- logger = logging.getLogger(__name__)
+ logger = structlog.get_logger()
```

**Only the changed parts appear in the diff = easier reviews**

With AST-based approaches, entire files get rewritten, leading to massive diffs

# 04. AI-Powered Rule Generation

Generating Custom Linter Rules from Natural Language

# Writing Fixit rules is surprisingly challenging

Requires understanding libcst's Visitor pattern and CST node structure

# The Traditional Barrier to Rule Creation

```
# You need to understand libcst's node structure
cst.Import(
    names=[
        cst.ImportAlias(
            name=cst.Attribute(
                value=cst.Name("logging"),
                attr=cst.Name("getLogger"),
            ),
        ),
    ],
)
```

- Deep knowledge of CST node types and hierarchy is required
- Understanding `visit_*` / `leave_*` method usage in the Visitor pattern
- Writing safe node modifications with `with_changes()`
- → **High learning curve, can't write rules right away**

# AI-Powered Rule Generation Flow

1. **Describe in Natural Language**

   "Replace logging.getLogger usage with structlog.get_logger"

2. **AI Generates the Fixit Rule**

   Automatically generates a LintRule class based on libcst's Visitor pattern

3. **Apply Across the Entire Codebase**

   Bulk autofix with the fixit fix command

# AI Lowers the Barrier

## An era where anyone can create custom linters

Even without deep knowledge of libcst, simply describing what you want in natural language automatically generates Fixit rules.
This enables the entire team to participate in defining and enforcing coding standards.

# Tips for Providing Context to AI

```
# Information that improves accuracy when included in prompts

1. Basic structure of fixit.LintRule (template)
2. List of key libcst node types
3. Before / After transformation examples
4. Specific edge case examples
5. Implementation examples of existing Fixit rules
```

**AI excels at pattern matching → better examples lead to better accuracy**

Passing existing Fixit built-in rules as reference examples is also effective

# 05. Practice

Automating the logging → structlog Migration

# Why structlog?

## Industry Trend Toward Structured Logging

Python's standard logging module is flexible but cumbersome for structured log output.

structlog enables concise key-value format logging with excellent compatibility with JSON output and log pipelines.

Many organizations are making the switch.

# The Transformation Goal

Before:

```
import logging

logger = logging.getLogger(__name__)
logger.info("User logged in", extra={"user_id": user_id})
```

After:

```
import structlog

logger = structlog.get_logger()
logger.info("User logged in", user_id=user_id)
```

We want to automatically handle **both** the import change and the function call change

# Example Prompt for AI

```
Please generate the following Fixit rule:

1. Transform import logging to import structlog
2. Transform logging.getLogger(__name__) to structlog.get_logger()
3. Preserve existing comments and formatting

The rule should inherit from fixit.LintRule,
detect using visit_ImportFrom / visit_Call methods,
and implement autofix.
```

# AI-Generated Fixit Rule (1/3): Class Definition

```
import libcst as cst
from libcst import matchers as m
import fixit

class ReplaceLoggingWithStructlog(fixit.LintRule):
    """Rule to replace logging module usage with structlog"""

    MESSAGE = "Use structlog instead of logging"
    TAGS = {"migration", "logging"}
```

Simple class definition and metadata declaration

# AI-Generated Fixit Rule (2/3): Import Detection

```
    def visit_Import(self, node: cst.Import) -> None:
        """Detect import logging and transform to import structlog"""
        if isinstance(node.names, (list, tuple)):
            for i, alias in enumerate(node.names):
                if isinstance(alias.name, cst.Name) \
                   and alias.name.value == "logging":
                    new_names = list(node.names)
                    new_names[i] = alias.with_changes(
                        name=cst.Name("structlog")
                    )
                    new_node = node.with_changes(
                        names=new_names
                    )
                    self.report(node, replacement=new_node)
```

`with_changes()` replaces the node while preserving comments and whitespace

# AI-Generated Fixit Rule (3/3): Function Call Detection

```
    def visit_Call(self, node: cst.Call) -> None:
        """Detect logging.getLogger(__name__)"""
        if self._is_logging_get_logger(node):
            new_node = cst.Call(
                func=cst.Attribute(
                    value=cst.Name("structlog"),
                    attr=cst.Name("get_logger"),
                ),
                args=[],  # No arguments needed for structlog
            )
            self.report(node, replacement=new_node)

    def _is_logging_get_logger(self, node: cst.Call) -> bool:
        return (
            isinstance(node.func, cst.Attribute)
            and isinstance(node.func.value, cst.Name)
            and node.func.value.value == "logging"
            and node.func.attr.value == "getLogger"
        )
```

# Testing the Rule

```python
# tests/test_replace_logging.py
from fixit import LintRuleTest

class TestReplaceLogging(LintRuleTest):
    RULE = ReplaceLoggingWithStructlog

    VALID = [
        # Correct code using structlog
        "import structlog\nlogger = structlog.get_logger()",
    ]

    INVALID = [
        # Violating code using logging
        "import logging\nlogger = logging.getLogger(__name__)",
    ]
```

**Fixit includes a built-in test framework** → ensuring rule quality

# Bulk Application with Fixit Commands

```
# Lint run (detection only)
$ fixit lint src/
src/auth/service.py:3:1 ReplaceLoggingWithStructlog:
  Use structlog instead of logging
src/api/handler.py:5:1 ReplaceLoggingWithStructlog:
  Use structlog instead of logging
...Found 23 violations in 23 files.

# Apply autofix in bulk
$ fixit fix src/
Fixed 23 files.

# Interactive one-by-one review
$ fixit fix --interactive src/
```

Migration of 23 files completed in seconds

# Traditional Approach vs Fixit + AI

### Traditional Approach

- Manually search and replace across all files
- Regex struggles with complex patterns
- Risk of breaking comments and formatting
- Requires expert knowledge to create rules

### Fixit + AI

- Safe automatic transformation with CST
- Accurately detects complex syntax patterns
- Fully preserves comments and formatting
- Generate rules from natural language

**AI + CST enables "safe," "fast," and "anyone can create" custom linters**

# Part 1 Summary: Impact

**Minutes**

Rule Creation Time

Generated from natural language with AI

**Seconds**

Fix Entire Codebase

Bulk application with fixit fix

**100%**

Formatting & Comment Retention

Safe transformation via CST

# Part 2: Fixit Linter for AI Coding

How to Safeguard the Quality of AI-Generated Code

# 06. Quality Challenges of AI-Generated Code

New Risks in the AI Coding Era

How much do you

trust the code

written by AI?

# The Light and Shadow of AI Coding

## Benefits of AI Coding

- Dramatic improvement in code generation speed
- Automatic boilerplate generation
- Instant use of unfamiliar libraries
- Code generation from natural language

## Often Overlooked Risks

- Code unaware of organizational conventions
- Introduction of deprecated patterns
- Tests that "just pass" without real quality
- Security concerns

# Problematic Code AI Tends to Generate

```python
# AI generates working code, but doesn't know your org's conventions

import logging                      # ← Should use structlog
import requests                     # ← Should use internal HTTP client
from datetime import datetime

logger = logging.getLogger(__name__)

def get_user(user_id: str):
    response = requests.get(f"/api/users/{user_id}")
    created_at = datetime.now()   # ← Testability issue
    logger.info(f"Fetched user {user_id}")  # ← Should use structured logging
    return response.json()
```

It works, but doesn't meet the team's standards

# Why Custom Linters Matter in the AI Era

## AI generates code. Linters correct code.

As the volume of AI-generated code increases, **automated enforcement of organization-specific quality standards** becomes essential.
Fixit's custom rules can be **freely customized to match each organization's or team's standards**, enabling **instant feedback on AI-generated code, just like a human reviewer**.

# 07. Guarding AI Code Quality with Fixit

Applying Custom Rules to Generated Code

# AI Coding + Fixit Workflow

1. **AI Generates Code**

   Generate code with Copilot / ChatGPT / Claude, etc.

2. **Fixit Checks Instantly**

   Custom rules automatically detect violations of organizational standards

3. **Autofix or Notify Developer**

   Autofix with fixit fix, or block the PR in CI

# Applying Fixit to AI-Generated Code

```
# Check AI-generated code with fixit
$ fixit lint src/ai_generated/

src/ai_generated/user_service.py:1:1 ReplaceLoggingWithStructlog
  Use structlog instead of logging
src/ai_generated/user_service.py:8:5 BanDatetimeNow
  Direct use of datetime.now() is prohibited
src/ai_generated/user_service.py:3:1 BanRequestsLibrary
  Use internal HTTP client instead of requests

Found 3 violations in 1 file.

# Bulk autofix
$ fixit fix src/ai_generated/
Fixed 1 file.
```

Automatically correct AI-generated output to meet team standards

# Fixit Rule Examples for AI-Generated Code

## Detecting Deprecated Libraries

AI tends to use requests, urllib3, etc. from outdated information → auto-convert to internal client

## Enforcing Security Patterns

Prohibit eval() / exec() / pickle.loads(), detect SQL string concatenation

## Correcting Coding Conventions

Naming conventions, mandatory type annotations, enforced docstrings, and other organizational rules

# 08. mutmut

Strengthening Fixit Rule Test Cases

# What Is mutmut

## mutmut - Python Mutation Testing

A tool that introduces intentional small changes (mutants) to code and verifies whether tests can detect them. It measures the "true quality" of your tests.

- Measures **test detection capability** that code coverage alone cannot reveal
- Fast mutation verification through parallel execution
- Interactive UI to browse mutant results
- Incremental execution (resume from where you left off)

# Are the VALID / INVALID cases in your Fixit rules really enough?

Mutation testing reveals gaps in your test cases

# A Quick Review of Fixit Rule Testing

```
class TestReplaceLogging(LintRuleTest):
    RULE = ReplaceLoggingWithStructlog

    VALID = [
        "import structlog",                          # Correct import
    ]

    INVALID = [
        "import logging",                            # Violating import
    ]
```

**Looks sufficient at first glance, but... what about these cases?**

- What about `from logging import getLogger`?
- What about `import logging as log`?
- What about `import logging, os`?

# Coverage vs Mutation Testing

## Code Coverage

- Measures "lines executed by tests"
- Can reach 100% by just executing lines
- VALID/INVALID completeness is unknown
- Cannot detect missing rule conditions

## Mutation Testing

- Mutates the rule's code for verification
- Checks if VALID/INVALID can detect mutations
- Discovers missing test cases
- Reveals the true robustness of rules

# What mutmut Does to Fixit Rules

```
# Original rule
def _is_logging_get_logger(self, node):
    return (
        isinstance(node.func, cst.Attribute)
        and node.func.value.value == "logging"     # ← Mutated here
        and node.func.attr.value == "getLogger"    # ← Mutated here too
    )
```

Mutants generated by mutmut:

```
# Mutant 1: Change the string
and node.func.value.value == "XXloggingXX"   # "logging" changed

# Mutant 2: Remove the condition
and True                                      # Condition always True

# Mutant 3: Change and to or
or node.func.attr.value == "getLogger"        # and → or
```

If VALID/INVALID can't detect these mutations, test cases are insufficient

# How to Use mutmut

```
# Run mutation testing on Fixit rules
$ mutmut run --paths-to-mutate=lint_rules/

# Check results
$ mutmut results
Survived ☹️: 3    ← Mutations not detected by VALID/INVALID
Killed ✓:   12   ← Correctly detected mutations
Total:      15
Mutation score: 80%

# View details of survived mutants
$ mutmut browse
```

Survived = test cases missing from VALID/INVALID

# Adding Test Cases from Survived Mutants

```
# mutmut found: "from logging import ..." not detected
# → Add cases to INVALID

class TestReplaceLogging(LintRuleTest):
    RULE = ReplaceLoggingWithStructlog

    VALID = [
        "import structlog",
        "from structlog import get_logger",      # ✅ Added
        "import logging_utils",                   # ✅ Added: similar name allowed
    ]

    INVALID = [
        "import logging",
        "from logging import getLogger",          # ✅ Added
        "import logging as log",                  # ✅ Added
        "import os, logging",                     # ✅ Added
    ]
```

mutmut Survived → add test cases → improve rule robustness

# Fixit Rule Strengthening Flow with mutmut

1. **AI Generates Fixit Rule + VALID/INVALID**

   Auto-generate basic rules and test cases from natural language

2. **mutmut Mutation-Tests the Rule Code**

   Detect which parts of the rule implementation aren't covered by VALID/INVALID

3. **Strengthen VALID/INVALID from Survived Mutants**

   Add missing edge cases to tests, ensuring rule quality

# The Fixit + mutmut Combination

## Protect rule quality with the rule's own tests

**Fixit** = Enforce code quality with custom rules
**mutmut** = Verify the completeness of those rules' VALID/INVALID test cases

AI-generated rules and test cases can "work but not be sufficient." mutmut **automatically detects gaps in rule test cases**, enabling robust custom linters.

# 09. CI/CD Integration & Operations

How to Adopt Across Your Team

# Integrating into the CI/CD Pipeline

```yaml
# .github/workflows/quality.yml
name: AI Code Quality Gate
on: [pull_request]

jobs:
  fixit:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
      - run: pip install fixit mutmut
      - run: fixit lint src/                # Code quality check
      - run: mutmut run --CI               # Test quality check
```

Automatically verify code quality + test quality on every PR

# Summary

# Thank You!

Automatically safeguard your team's
code quality with Fixit + AI

GitHub: github.com/naohide

LinkedIn: linkedin.com/in/naohide